



TEXAS A&M UNIVERSITY - SAN ANTONIO

Chapter 4

The Efficiency of Algorithms

Spring 2026



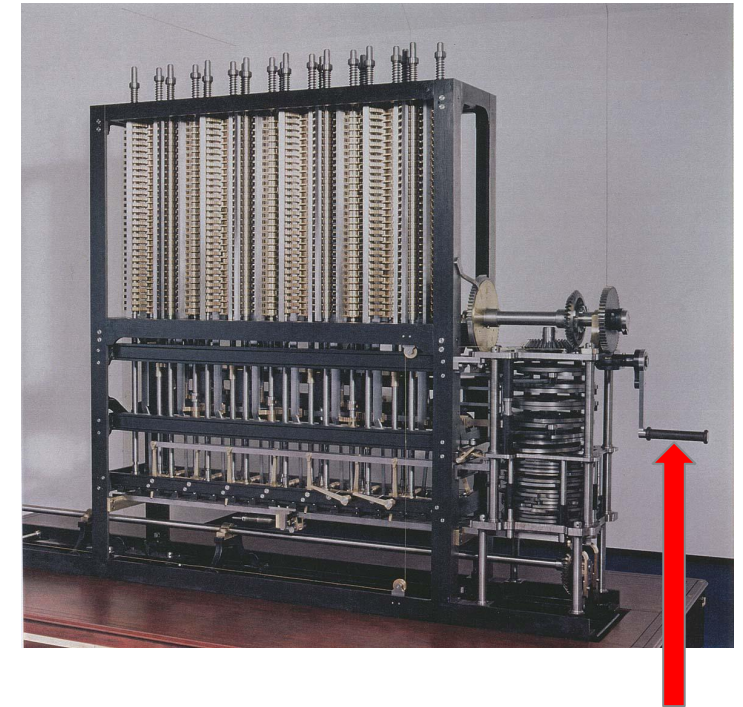
Objectives

- Measure the efficiency of a given algorithm
- Compare the expected execution times of different methods, given the efficiencies of their algorithms



Why Efficient Code?

- Computers are faster, have larger memories
 - So why worry about efficient code?
 - Predict performance
 - Compare various approaches to solve a problem
 - Provide guarantees



how many times do you have to turn the crank?



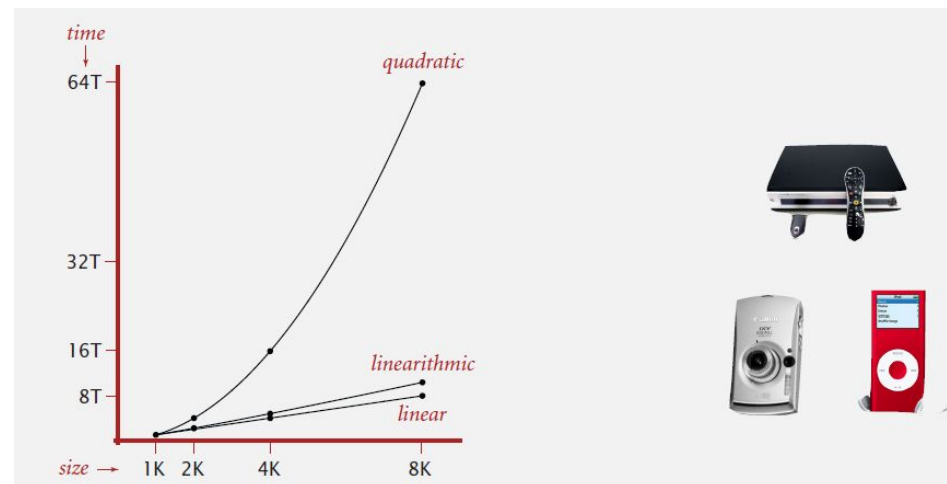
Why Efficient Code?

- An algorithm has both **time** and **space** constraints – that is **complexity**
 - **Time complexity** – time it takes to execute
 - **Space complexity** – memory it needs to execute
 - A best one might be the **fastest** one or the one that uses the **least memory**



Some algorithmic successes

- Discrete Fourier Transform
 - Applications: DVD, JPEG, MRI, astrophysics,
 - Break down waveform of N samples into periodic components.
 - Brute force: N^2 steps
 - FFT algorithm: $N \log N$ steps, **enables new technology.**





Analysis Of Algorithms

- Process of measuring the complexity of algorithms
- Concentrate on the **time** complexity
- It is usually more important than the **space** complexity
- Inverse relations often exist over those two
 - To save time, will usually need more memory space



Importance of Efficiency

- Consider the following problem:

summing $1 + 2 + \dots + n$ for an integer $n > 0$

Algorithm A	Algorithm B	Algorithm C
<pre>long sum = 0; for (long i = 1; i <= n; i++) sum = sum + i;</pre>	<pre>sum = 0; for (long i = 1; i <= n; i++) { for (long j = 1; j <= i; j++) sum = sum + 1; }</pre>	<pre>sum = n * (n + 1) / 2;</pre>

Which one is more time efficient?



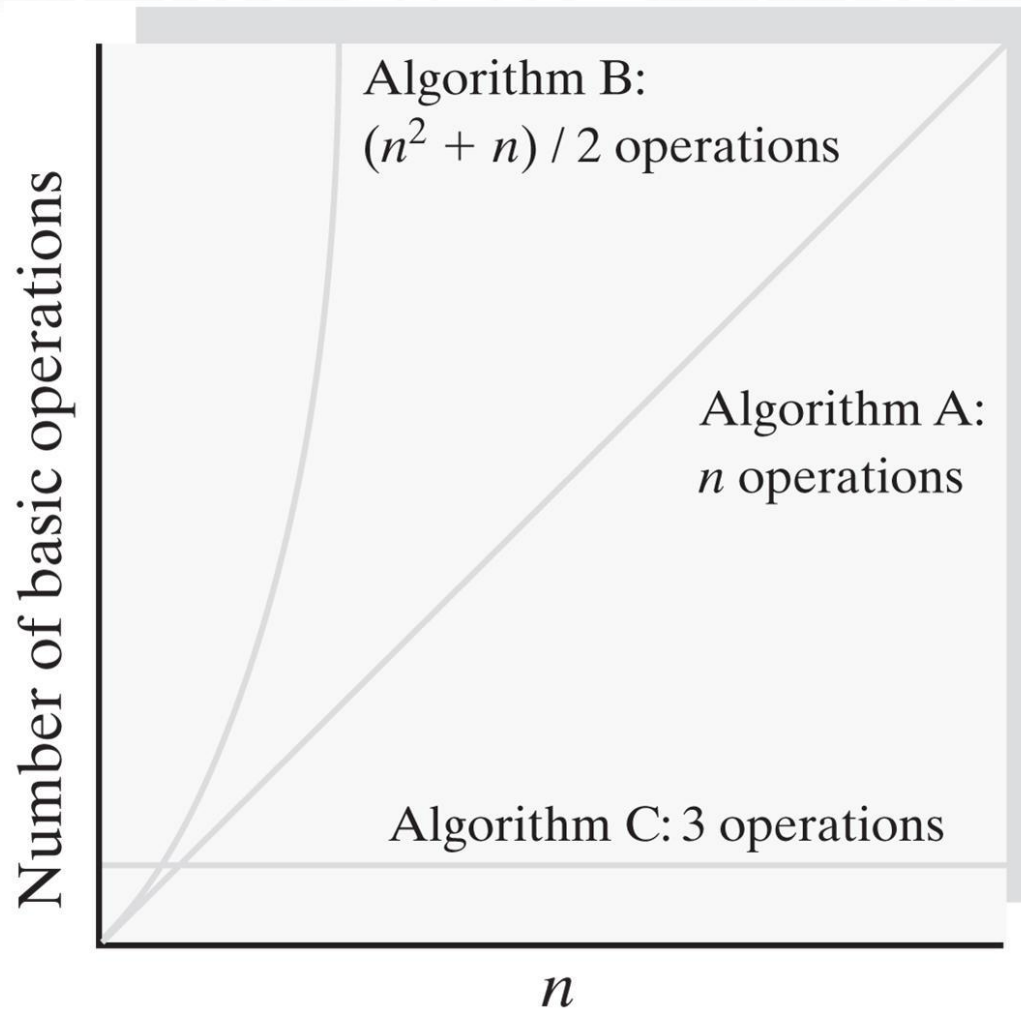
Counting Basic Operations

- A **basic operation** of an algorithm: most significant contributor to its total time requirement

	Algorithm A	Algorithm B	Algorithm C
	<pre>long sum = 0; for (long i = 1; i <= n; i++) sum = sum + i;</pre>	<pre>sum = 0; for (long i = 1; i <= n; i++) { for (long j = 1; j <= i; j++) sum = sum + 1; }</pre>	<pre>sum = n * (n + 1) / 2;</pre>
Additions	n	$n(n + 1)/2$	1
Multiplications	0	0	1
Divisions	0	0	1
Total Basic Operations	n	$(n^2 + n)/2$	3



Counting Basic Operations



Number of basic operations required by the algorithms as a function of **n**



Picturing Efficiency

```
long sum = 0;  
for (long i = 1; i <= n; i++)  
    sum = sum + i;
```



1



2



3

...



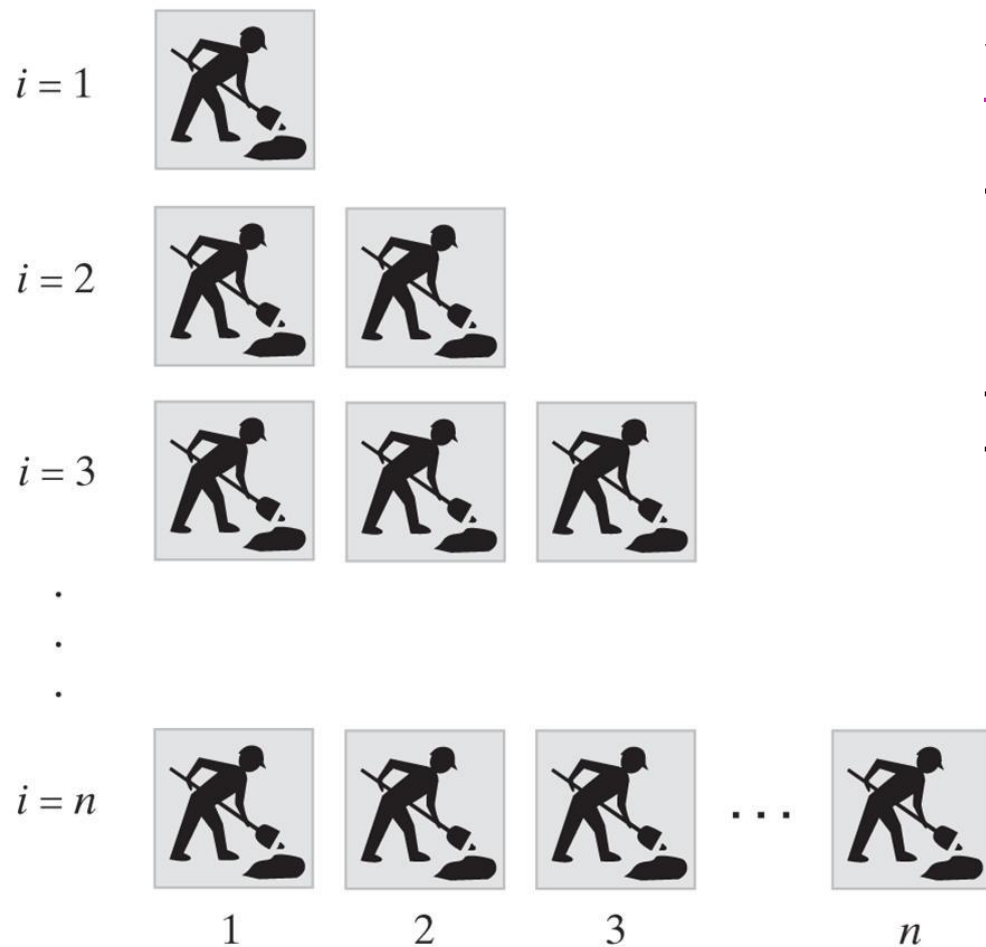
n

$O(n)$

© 2019 Pearson Education, Inc.



Picturing Efficiency



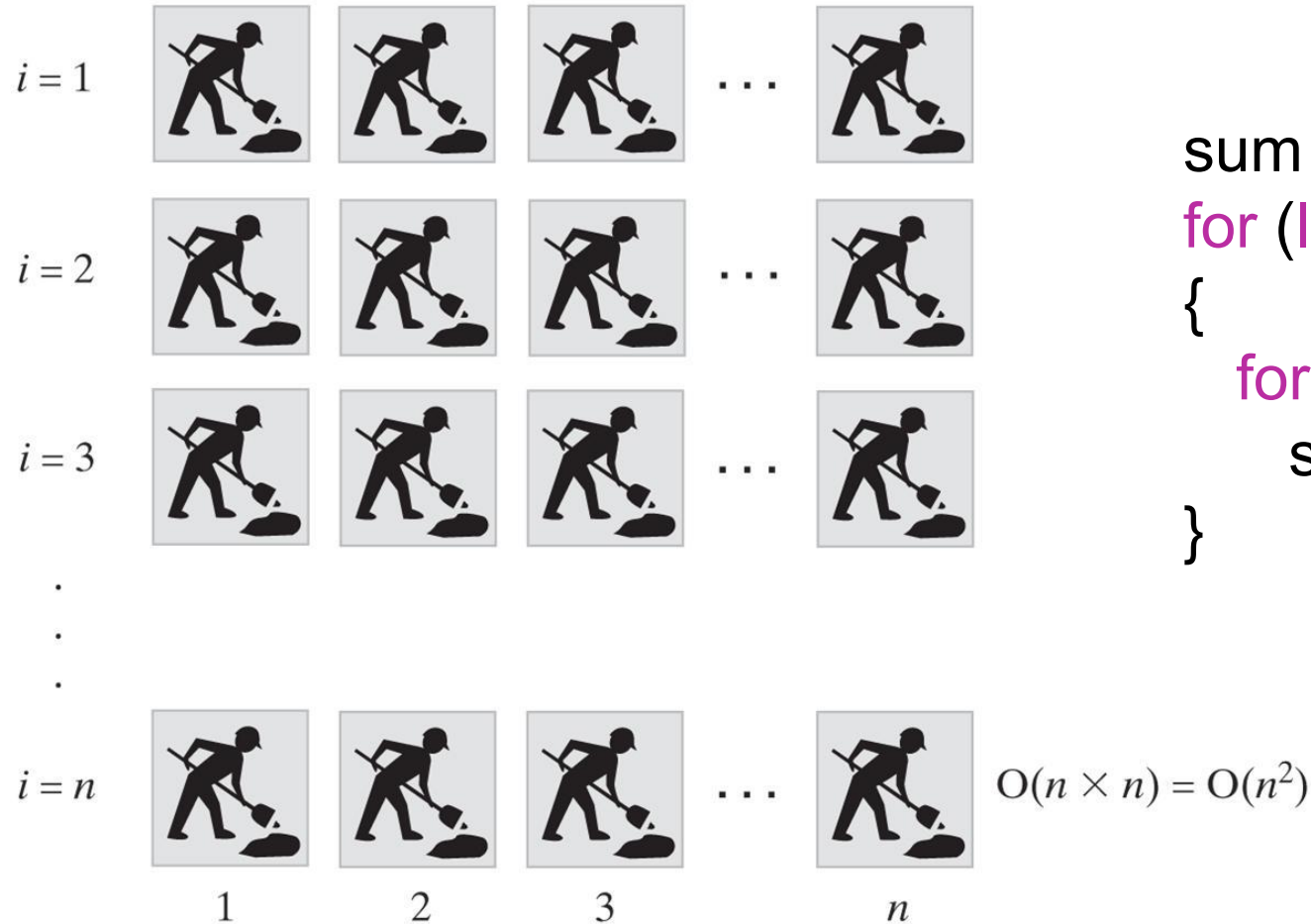
```
sum = 0;  
for (long i = 1; i <= n; i++)  
{  
    for (long j = 1; j <= i; j++)  
        sum = sum + 1;  
}
```

change to n ?

$$O(1 + 2 + \dots + n) = O(n^2)$$



Picturing Efficiency



```
sum = 0;
for (long i = 1; i <= n; i++)
{
    for (long j = 1; j <= n; j++)
        sum = sum + 1;
}
```

© 2019 Pearson Education, Inc.



Asymptotic Complexity

- The resulting function gives only an **approximate** measure of efficiency of the original function.
 - Any terms that do not substantially change the function's magnitude should be eliminated.
 - This approximation is sufficiently close to the original function.
 - This measure of efficiency is called **asymptotic complexity**.
- Used
 - To express the efficiency of an algorithm
 - When calculating a function is difficult or impossible and only approximations can be found



Asymptotic Complexity with Big O

n	$\log(\log n)$	$\log n$	$\log^2 n$	n	$n \log n$	n^2	n^3	2^n	$n!$
10	2	3	11	10	33	10^2	10^3	10^3	10^5
10^2	3	7	44	100	664	10^4	10^6	10^{30}	10^{94}
10^3	3	10	99	1000	9966	10^6	10^9	10^{301}	10^{1435}
10^4	4	13	177	10,000	132,877	10^8	10^{12}	10^{3010}	$10^{19,335}$
10^5	4	17	276	100,000	1,660,964	10^{10}	10^{15}	$10^{30,103}$	$10^{243,338}$
10^6	4	20	397	1,000,000	19,931,569	10^{12}	10^{18}	$10^{301,030}$	$10^{2,933,369}$

© 2019 Pearson Education, Inc.

The growth rate of function f depends mainly on the leading-order term, i.e., n^2 .

$$f(n) = n^2 + 100n + \log_{10} n + 1,000 = \mathbf{O(n^2)}$$



Asymptotic Complexity with Big O

The Computer Time Used by Algorithms.						
Problem Size	Bit Operations Used					
n	$\log n$	n	$n \log n$	n^2	2^n	$n!$
10	3×10^{-11} s	10^{-10} s	3×10^{-10} s	10^{-9} s	10^{-8} s	3×10^{-7} s
10^2	7×10^{-11} s	10^{-9} s	7×10^{-9} s	10^{-7} s	4×10^{11} yr	*
10^3	1.0×10^{-10} s	10^{-8} s	1×10^{-7} s	10^{-5} s	*	*
10^4	1.3×10^{-10} s	10^{-7} s	1×10^{-6} s	10^{-3} s	*	*
10^5	1.7×10^{-10} s	10^{-6} s	2×10^{-5} s	0.1 s	*	*
10^6	2×10^{-10} s	10^{-5} s	2×10^{-4} s	0.17 min	*	*

*Times of more than 10^{100} years are indicated with an **

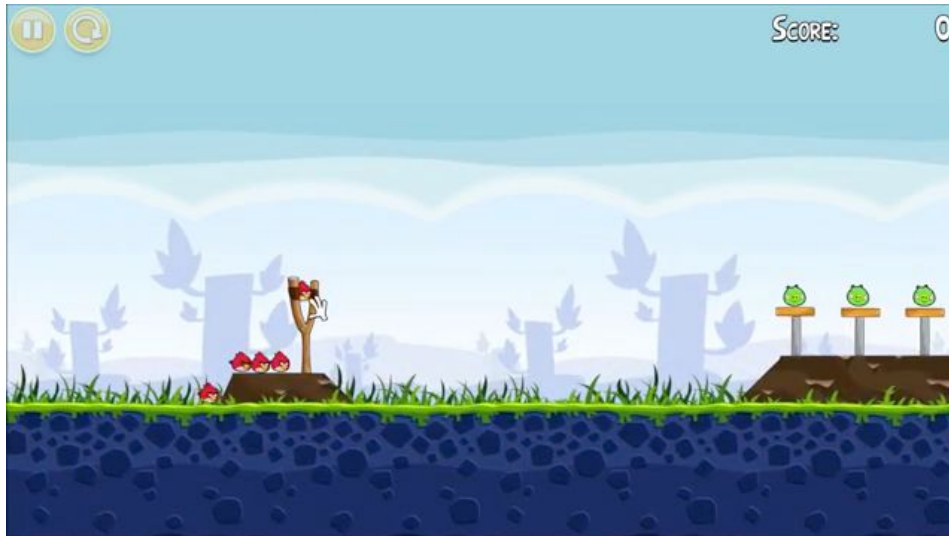


3-Sum Problem

- Given N distinct integers, how many triples sum to exactly zero?

Context: deeply related to problems in computational geometry

```
% more 8ints.txt  
8  
30 -40 -20 -10 40 0 10 5
```





3-Sum Brute-Force Algorithm

```
public class ThreeSum
{
    public static int count(int[] a)
    {
        int N = a.length;
        int count = 0;
        for (int i = 0; i < N; i++)
            for (int j = i+1; j < N; j++)
                for (int k = j+1; k < N; k++)
                    if (a[i] + a[j] + a[k] == 0)
                        count++;
        return count;
    }

    public static void main(String[] args)
    {
        In in = new In(args[0]);
        int[] a = in.readAllInts();
        StdOut.println(count(a));
    }
}
```

← check each triple

← for simplicity, ignore
integer overflow

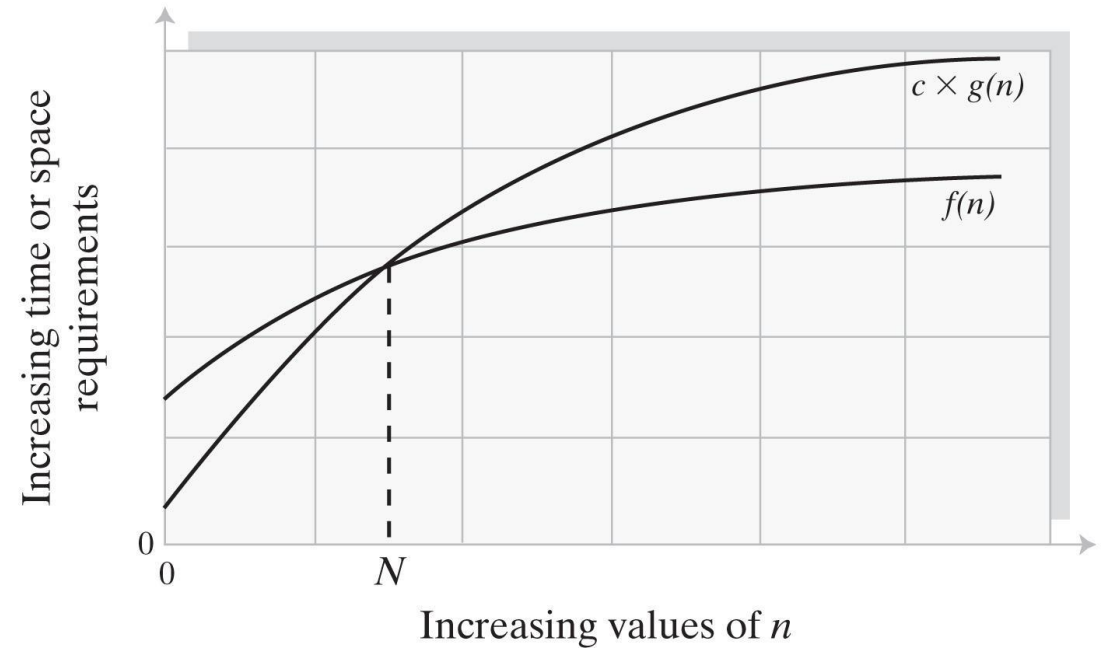
time complexity?

$$O(n^3)$$



Big O Notation

- A function $f(n)$ is of order **at most** $g(n)$
- That is, $f(n)$ is $O(g(n))$ — **if:**
 - A positive real number c and positive integer N exist ...
 - Such that $f(n) \leq c \times g(n)$ for all $n \geq N$
 - That is:
 - $c \times g(n)$ is an **upper bound** on $f(n)$ when n is sufficiently large





Big O Notation

Definition: $f(n)$ is $O(g(n))$ if there exist positive numbers c and N such that $f(n) \leq c g(n)$ for all $n \geq N$

• **Example:** Show that $f(x) = x^2 + 2x + 1$ is $O(x^2)$

Solution: Since when $x \geq 1$, $x \leq x^2$ and $1 \leq x^2$

Can take $c = 4$ and $N = 1$

Alternatively, when $x \geq 2$, we have $2x \leq x^2$ and $1 \leq x^2$. Hence,

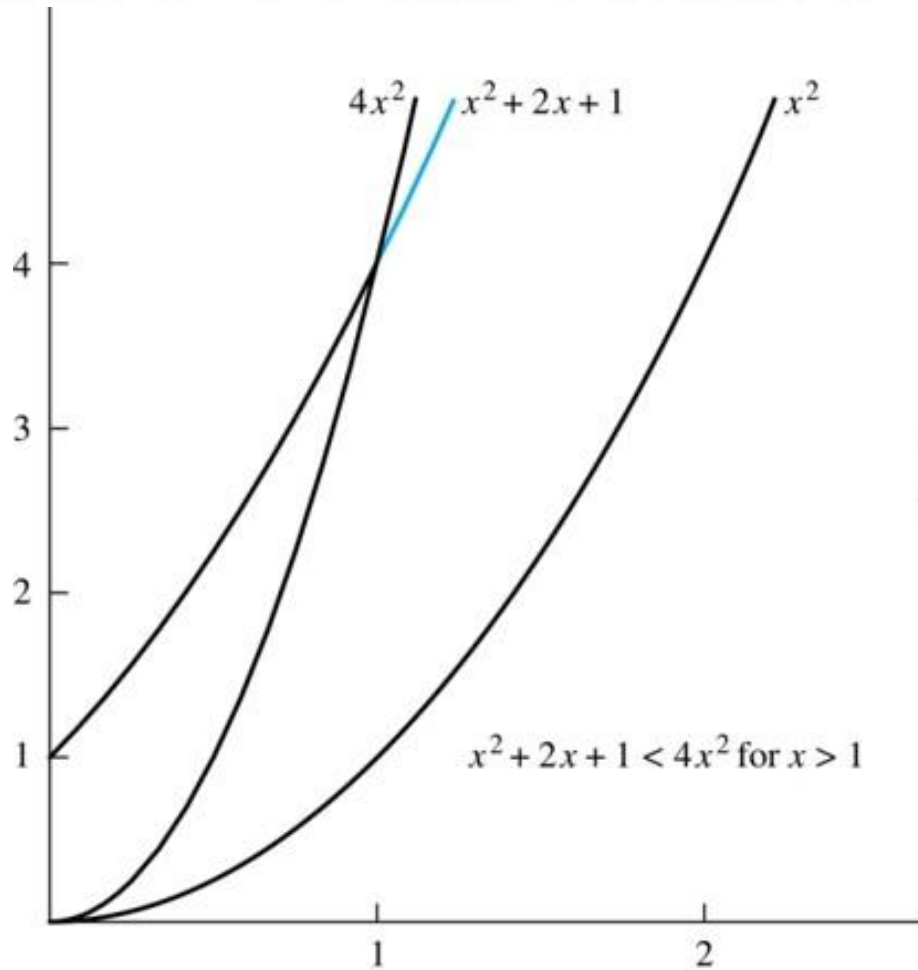
Can take $c = 3$ and $N = 2$

...

...



Big O Notation



The part of the graph of $f(x) = x^2 + 2x + 1$ that satisfies $f(x) < 4x^2$ is shown in blue.

$$x^2 + 2x + 1 < 4x^2 \text{ for } x > 1$$



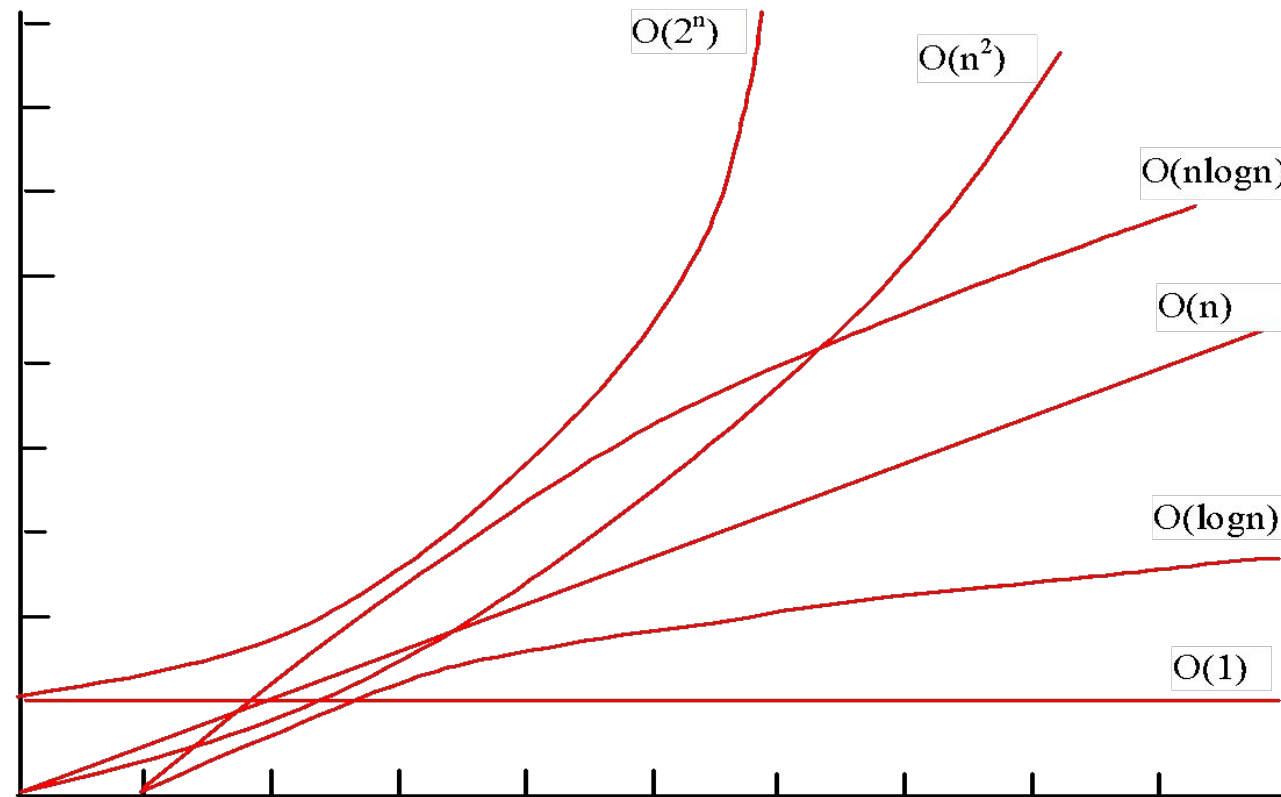
Picturing Efficiency

order of growth	name	typical code framework	description	example
1	constant	<code>a = b + c;</code>	statement	add two numbers
$\log N$	logarithmic	<pre>while (N > 1) { N = N / 2; ... }</pre>	divide in half	binary search
N	linear	<pre>for (int i = 0; i < N; i++) { ... }</pre>	loop	find the maximum
$N \log N$	linearithmic	mergesort	divide and conquer	mergesort
N^2	quadratic	<pre>for (int i = 0; i < N; i++) for (int j = 0; j < N; j++) { ... }</pre>	double loop	check all pairs
N^3	cubic	<pre>for (int i = 0; i < N; i++) for (int j = 0; j < N; j++) for (int k = 0; k < N; k++) { ... }</pre>	triple loop	check all triples
2^N	exponential	combinatorial search	exhaustive search	check all subsets



Comparing Common Growth Functions

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n)$$





Question

- What is the asymptotic complexity (big O) of function

$$f(n) = 5n^2 + 2^n + 10n$$